UNIVERSITI MALAYSIA SARAWAK

FACULTY OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

TMA3084 SOFTWARE ENGINEERING LAB

GROUP PROJECT ASSESSMENT GUIDELINES

Semester 1, 2026/2027

Final submission: 2 January 2027, 10:00 PM

Document purpose

This guideline consolidates the project scope, team responsibilities, technical baseline, AI-assisted engineering controls, TSP evidence, milestone deliverables, assessment rubric, and final submission requirements into one operational reference.

Team size	4–5 students	**Project duration**	15 weeks
Primary platform	Flutter / Dart	**AI requirement**	≥1 cloud Generative AI feature
Process evidence	GitHub + TSP logs	**Assessment**	4 milestones = 100%

Use this document together with the current lecture notes, project forms, and LMS announcements.

Contents

Assessment at a Glance 1
1. Course Overview & Academic Scope 1
2. Team Formation & Collaboration Policy 1
2.1 Group Composition & Accountability 1
2.2 Geolocation Radius Grouping 2
3. Technical Stack & Architecture Baseline 3
4. Spec-Driven AI-Assisted Development Workflow 3
4.1 Non-Negotiable Controls 3
4.2 AI Usage Documentation Log (LOGAI) 4
5. TSP Process Rigor, Evidence & Traceability 4
6. Project Milestones & Assessment Breakdown 5
7. Detailed Assessment Rubric 6
8. Submission Instructions & Penalties 7
8.1 Required Repository / Submission Structure 7
8.2 Late Submission Penalty 7
8.3 Final Submission Checklist 7
9. Academic Integrity & Ethics 7
10. References 8

Assessment at a Glance

Area	Minimum expectation	Primary evidence
Team	4–5 students; each has a primary accountability area	FORM-SI, team charter, peer evaluation
Stakeholder	Real client/stakeholder with active validation touchpoints	Client contact evidence, scope sign-off, UAT, acceptance
Mobile app	Cross-platform Flutter 3.13+ / Dart 3.0+	Runnable builds, repository, demonstration
Architecture	State management + cloud backend/API integration	SDD, architecture diagrams, source structure
AI	At least one cloud Generative AI API feature	Implemented feature, LOGAI, tests, demo
Collaboration	GitHub issues/projects, branches, PR review before merge	Issues, milestones, PRs, review history
Quality	Unit/widget/integration/UAT evidence; M3 target ≥50 test cases and ≥60% coverage	LOGTEST, test reports, CI workflow

1. Course Overview & Academic Scope

TMA3084 Software Engineering Lab is a capstone group project that connects academic software engineering principles with industrial-style practice. Each team progresses through the complete Software Development Life Cycle (SDLC): stakeholder discovery, requirements engineering, planning, architecture and design, implementation, verification and validation, deployment packaging, and client acceptance.

The course combines Team Software Process (TSP) discipline with contemporary mobile engineering. Projects use Flutter and Dart, GitHub-based collaboration, cloud services, and a controlled form of AI-assisted coding. The central expectation is not simply to produce a working application, but to produce a traceable body of engineering evidence showing how requirements, design decisions, code, tests, reviews, and stakeholder validation fit together.

Core assessment principle

Working software is necessary but not sufficient. Marks are awarded for product quality, engineering process discipline, responsible AI use, stakeholder validation, and documented teamwork.

2. Team Formation & Collaboration Policy

2.1 Group Composition & Accountability

Projects must be completed in teams of 4 to 5 students. Every member must hold a primary software engineering accountability area while contributing cross-functionally to development, review, testing, documentation, and client-facing activities. The five accountability areas are shown below. In a four-person team, the team must document in FORM-SI how one member covers two accountability areas while maintaining an equitable workload.

Primary role / accountability	Core responsibility	Typical assessable evidence
Software Project Manager (PM)	Coordinates architecture-level decisions, GitHub administration, milestone planning, risk escalation, and client communication.	Roadmap/milestones, meeting records, repository controls, client communication, integration decisions
Lead Systems Analyst & Requirements Engineer	Leads elicitation, SRS authoring, requirement IDs, acceptance criteria, change clarification, and traceability.	SRS, interview notes, requirement traceability, scope/sign-off records
Lead Mobile Architect (Flutter/Dart)	Leads UI/UX prototyping, state-management architecture, navigation structure, and Dart modularity.	SDD, prototypes, widget/module structure, design reviews
Backend & AI Integration Specialist	Leads backend/API integration, data schema, cloud AI connection, error/fallback handling, and integration evidence.	API/service modules, schema, AI feature, LOGAI, integration tests
Quality Assurance & CI/CD Engineer	Leads test planning and automation, CI/CD workflow, defect tracking, coverage evidence, and release checks.	LOGTEST, LOGD, CI runs, coverage report, release checklist

2.2 Geolocation Radius Grouping

Students are strongly encouraged to form teams whose members are within a practical physical radius when feasible. The purpose is operational: proximity makes stakeholder observation, in-person requirement elicitation, physical-device testing, and co-creation sessions easier to organise. Geolocation proximity is a collaboration preference rather than a substitute for documented engineering evidence.

Collaboration mode	Expected practice	Evidence
Co-located / local	Use in-person discovery, whiteboard design reviews, pair-programming sessions, and hands-on UAT where appropriate.	Meeting notes, photos/screenshots where permitted, signed feedback/UAT, GitHub evidence
Scattered / hybrid	Maintain asynchronous task tracking, online design/review sessions, hybrid stakeholder meetings, and remote test coordination.	GitHub Projects/Issues, meeting records, online sign-off, PR/review history
All teams	No merge to main without a Pull Request and at least one peer approval; all tasks must have explicit ownership and status.	PR approvals, issues, milestones, repository history

3. Technical Stack & Architecture Baseline

Teams must engineer a mobile-first solution using the standardized baseline below. The listed versions are minimums; teams should use the current stable release compatible with the course environment and document any dependency constraints in the repository.

Layer	Required baseline	Permitted / expected choices	Evidence to submit
Mobile SDK	Flutter 3.13+ / Dart 3.0+	Android and iOS capable; Material Design 3 conventions	pubspec.yaml, runnable build, responsive UI
State management	Architectural state separation	Provider, Riverpod, or GetX	SDD + consistent implementation
Backend / data	Cloud-backed data or service layer	Firebase, Supabase, REST/GraphQL, or custom Node.js/Python backend	Architecture/data design + integration evidence
AI service	≥1 cloud Generative AI API	Provider may vary; the selected model/service must be identified in LOGAI	Working feature, API integration, error handling, LOGAI
Version control	GitHub repository	Branches, issues/projects, milestones, PR workflow	Commit/PR/review history
Automation	CI/CD workflow	At minimum automated test execution on relevant PR/push events	Workflow file + successful runs

4. Spec-Driven AI-Assisted Development Workflow

AI-assisted coding is permitted and expected to be used transparently. The term “vibe coding” is used in the course to describe natural-language-guided code generation, but assessment requires a stricter spec-driven workflow: intended behaviour is defined first, AI-generated artefacts are reviewed by humans, tests verify the implementation, and the evidence is recorded.

Course operating principle

Specification first; AI assistance second. AI may generate code or other artefacts, but human engineers remain responsible for understanding, reviewing, refactoring, testing, and accepting every contribution that is merged into the product.

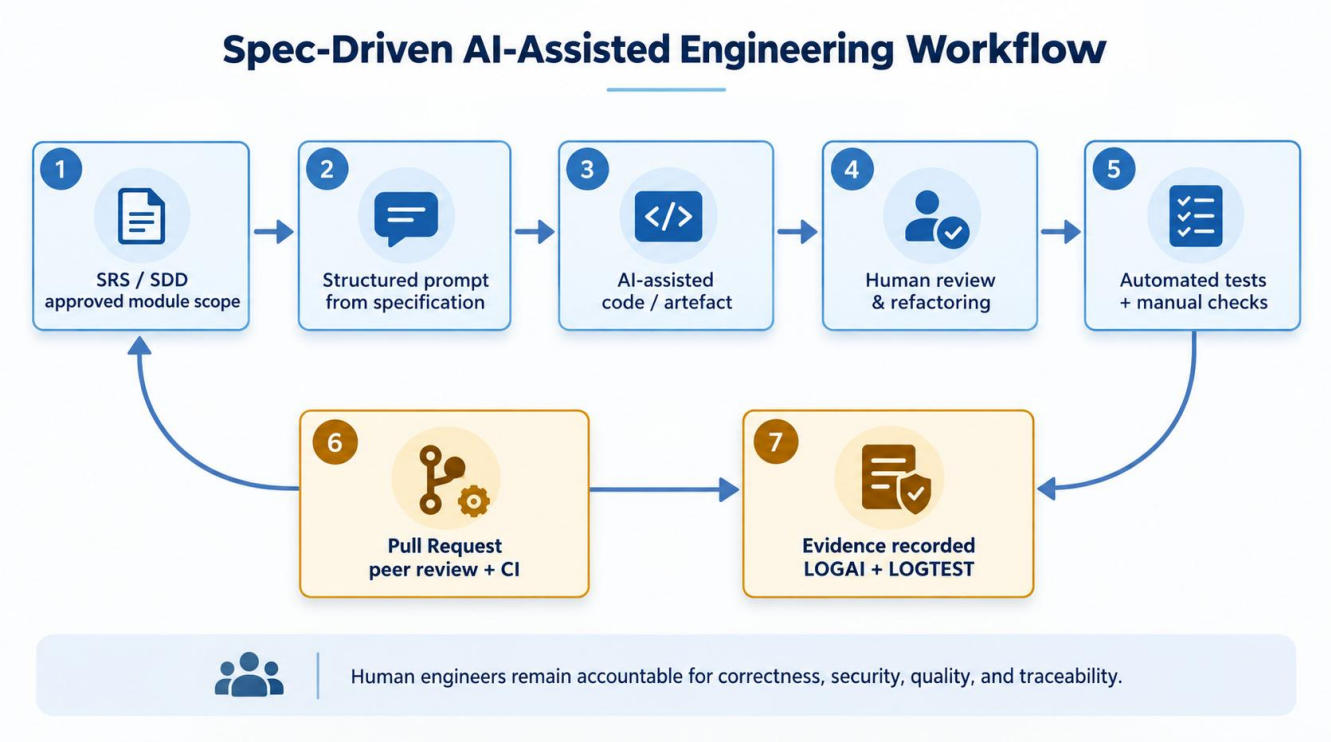


Figure 1. Required spec-driven AI-assisted engineering loop.

4.1 Non-Negotiable Controls

- Spec-first prompting: prompts for implementation work must be grounded in an SRS/SDD module, acceptance criteria, data model, interface contract, or equivalent project specification.

- Human-in-the-loop review: AI-generated code may not be merged without human review of the relevant diff for correctness, null safety, maintainability, performance, API use, and security concerns.
- Test-driven verification: AI-assisted modules must have appropriate automated tests and must be represented in LOGTEST; failures and fixes must be traceable.
- Evidence and traceability: significant AI-assisted work must be recorded in LOGAI and linked to the feature/module, test evidence, and corresponding GitHub Pull Request.

4.2 AI Usage Documentation Log (LOGAI)

LOGAI is a continuous project artefact, not an end-of-semester reconstruction. It provides transparency for AI use, human engineering effort, and verification. Each significant AI-assisted implementation or refactoring activity should contain the fields below.

LOGAI field	Requirement / description
Log ID & Date	Unique sequential identifier (for example, AI-001) and timestamp.
Target Feature / Module	Flutter widget, Dart class, state manager, API endpoint, test, or other project artefact being generated/refactored.
AI Tool & Model Used	Specific tool and underlying model/service used at the time of the interaction.
Specification / Prompt Provided	Prompt plus the relevant specification context, files, constraints, or instructions provided to the AI.
AI Output Summary	Concise description of the generated code, logic, test, documentation, or refactoring.
Human Refinement & Debugging	What the team reviewed, changed, corrected, optimised, or rejected and why.
Verification & GitHub PR Link	LOGTEST/test-run reference plus the corresponding PR or commit link.

5. TSP Process Rigor, Evidence & Traceability

Teams must implement the Team Software Process (TSP) discipline through regular planning, tracking, quality control, and reflection. Process forms should be updated as the work occurs so that the final report is supported by authentic project data.

Artefact	Purpose	Minimum use	Primary owner / contributors
FORM-SI	Establishes team membership, accountability areas, and contact channels.	Completed at launch; updated if responsibilities change.	PM + all members
LOGAI	Audits AI-assisted engineering and human verification.	Continuous; linked to significant AI-assisted work.	Feature owner + reviewer
LOGD	Records defect discovery, injection/removal phase, and fix effort.	Continuous during reviews/testing.	QA + all developers
LOGT	Records development effort by phase/task.	Continuous or at the reporting frequency taught in class.	All members
LOGTEST	Records unit, widget, integration, system/UAT execution and outcomes.	Every formal test execution used as evidence.	QA + test owner
WEEK	Summarises completed work, blockers, status, and next milestones.	Weekly after the first milestone.	PM / planning role + team
PEER	Provides confidential evidence of contribution and collaboration.	At required evaluation points and final assessment.	All members

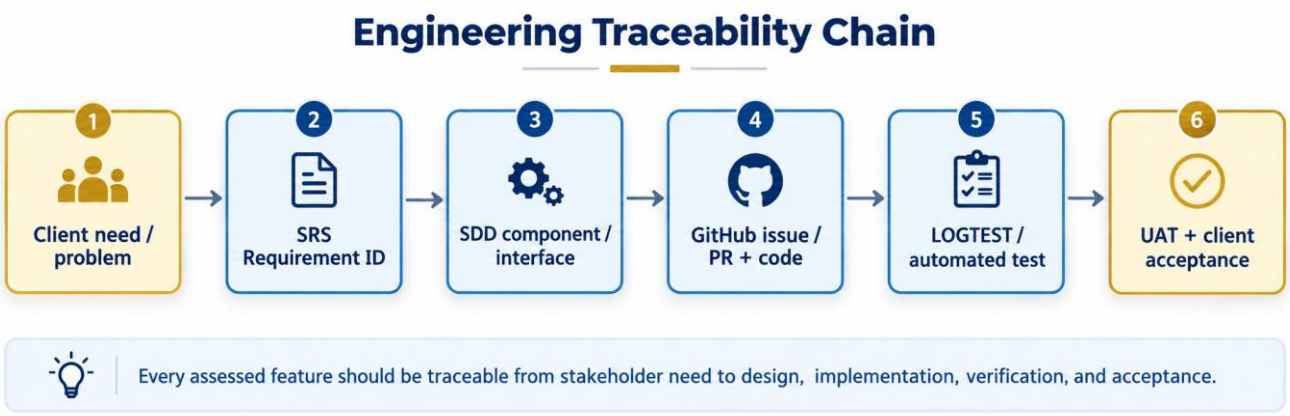


Figure 2. Traceability expected across requirements, design, implementation, verification, and stakeholder acceptance.

Traceability is particularly important for milestone reviews. A high-quality submission makes it possible to follow a requirement from the SRS into the SDD, the implementation issue/PR, the relevant code and tests, and finally the UAT or client acceptance evidence.

6. Project Milestones & Assessment Breakdown

The project spans 15 teaching weeks and is assessed through four milestones. The weights below sum to 100% of the group project assessment. Teams should treat each milestone as a quality gate: incomplete evidence at an early milestone increases integration and verification risk later.

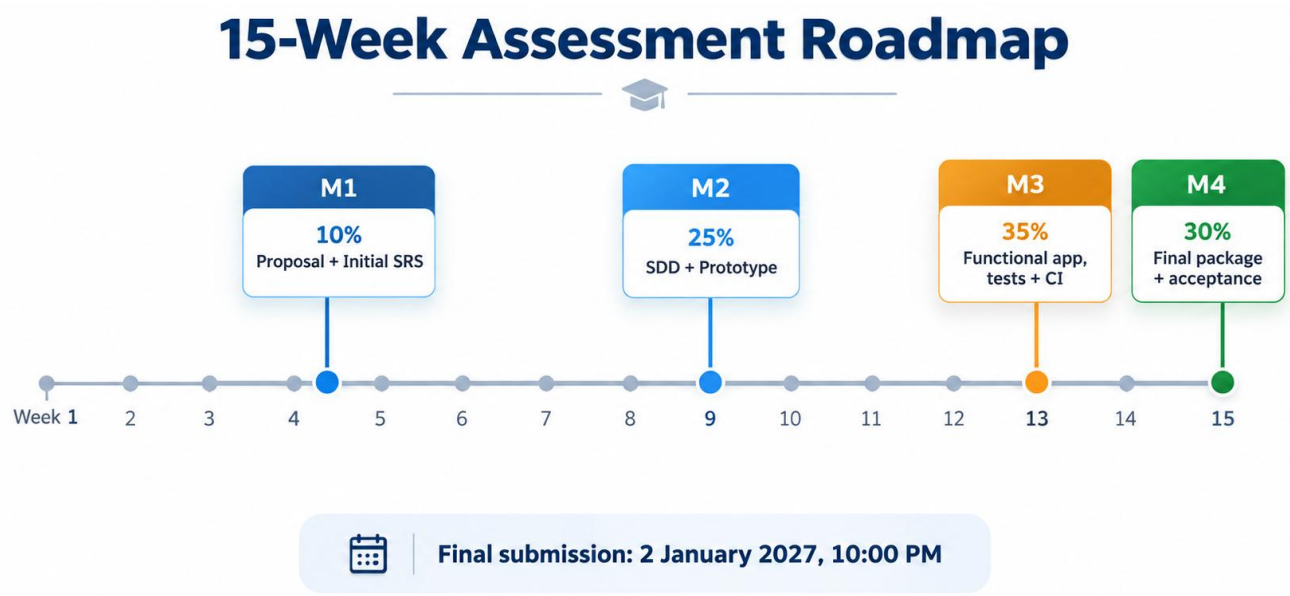


Figure 3. Milestone roadmap and assessment weights.

Milestone	Timeline / weight	Core deliverables	Quality gate
M1 – Scope & Requirements	Weeks 4–5 | 10%	Project proposal; initial SRS; team charter; client contact form; scope sign-off.	Problem and scope are credible; requirements are understandable, testable, and agreed.
M2 – Design & Prototype	Week 9 | 25%	SDD; runnable Flutter prototype with 4–5 key screens; initial LOGAI evidence (≥10 entries).	Architecture and interface decisions are coherent enough to support implementation.
M3 – Product & Verification	Week 13 | 35%	Functional Flutter app; integrated cloud AI feature; test suite (≥50 cases); ≥60% code coverage; GitHub CI/CD pipeline.	Key features work together; test and CI evidence demonstrates engineering quality.
M4 – Final Release & Acceptance	Week 15 / 2 Jan 2027 | 30%	Final report (50–80 pages); release APK/IPA build(s); user manual; 5–10 minute video demo; oral presentation; signed client acceptance form.	Final package is usable, documented, reproducible, and validated by the stakeholder.

7. Detailed Assessment Rubric

Evaluation is multi-dimensional. Strong performance requires both a credible product and a credible engineering process; one cannot fully compensate for the absence of the other.

Assessment category	Weight	What is assessed	High-standard evidence
Flutter App UX & Technical Quality	30%	Cross-platform responsiveness, Material Design 3 quality, null-safe Dart, modularity/state separation, reliability, and API error handling.	Consistent UX; stable navigation and state; maintainable code; graceful loading/error/empty states; reproducible build.
Software Process & GitHub Rigor	25%	Version control discipline, issue tracking, peer PR review, SRS/SDD completeness, and TSP evidence (LOGD, LOGT, LOGTEST, WEEK).	Meaningful commit/PR history; traceable issues; visible review quality; process logs match the repository timeline.
AI Feature & Vibe Coding Audit	20%	Value and integration of the cloud AI feature, LOGAI quality, human refinement, and verification of AI-assisted code.	AI feature solves a real use case; prompts/specification are documented; human decisions are visible; tests verify behaviour.
Stakeholder Engagement & Validation	15%	Physical/hybrid interactions, client feedback, UAT, and formal acceptance.	Repeated stakeholder touchpoints; actionable feedback; UAT evidence; signed FORM-CLIENT-ACCEPT or equivalent.
Presentation, Video & Teamwork	10%	Oral presentation clarity, 5–10 minute demo quality, peer evaluation, and equitable workload distribution.	Concise story from problem to evidence; reliable live/recorded demo; balanced contributions supported by PEER and repository history.

Assessment interpretation

Repository activity, logs, documents, tests, and stakeholder evidence should tell the same story. Last-minute documentation that cannot be reconciled with actual project history may not demonstrate the required process rigor.

8. Submission Instructions & Penalties

8.1 Required Repository / Submission Structure

All milestone deliverables and the final package must be submitted through the official UNIMAS LMS / GitHub Classroom channel specified by the teaching team. The final submission deadline is 2 January 2027 at 10:00 PM.

Location	Required contents	Submission check
/src	Complete source code, pubspec.yaml, test files, and .github/workflows/ci.yml.	Project builds; dependencies resolve; CI file is present.
/docs	PDF copies of final SRS, SDD, final report, user manual, and signed client acceptance form.	Files open correctly; final versions only; filenames are clear.
/process_forms	Completed LOGAI, LOGD, LOGT, LOGTEST, WEEK, and other required TSP evidence.	Entries are complete, dated, internally consistent, and traceable.

8.2 Late Submission Penalty

Delay after deadline	Penalty
On time	No late penalty
Up to 24 hours late	10% deduction
More than 24 and up to 48 hours late	20% deduction
More than 48 and up to 72 hours late	30% deduction

Submissions delayed beyond 72 hours receive a score of zero.

8.3 Final Submission Checklist

- Release build(s) are installable and correspond to the submitted source revision.
- All required documents and forms are final, readable, and use consistent requirement/test identifiers.
- GitHub main branch reflects the assessed release; PR, issue, milestone, and CI evidence remains accessible.
- Secrets, passwords, and private API keys are not included in the repository or report screenshots.
- Client acceptance evidence, video demo, and presentation materials are complete.

9. Academic Integrity & Ethics

AI tools are permitted only within the documented spec-driven workflow. Teams must disclose AI-assisted work through LOGAI and remain able to explain, modify, test, and defend the submitted implementation. External code, libraries, templates, and generated artefacts must be used in accordance with course rules, licences, and attribution requirements.

Permitted	Required	Not acceptable
AI-assisted coding, debugging, test generation, documentation support, and ideation.	LOGAI evidence; human review; relevant tests; team understanding of the final result.	Undeclared AI use presented as wholly unaided work.
Open-source packages and frameworks that are compatible with the project.	Licence/attribution compliance and documented dependency use.	Copying an external application/codebase and presenting it as original project work.
Collaborative peer review and pair programming.	Clear contribution history and equitable participation.	Fabricated logs, fabricated stakeholder evidence, or falsified test/coverage results.

Academic accountability

Submitting unverified AI-generated code without the required audit trail, or misrepresenting external work as original team work, is treated as an academic integrity issue under UNIMAS rules.

10. References

Google Developers. (2026). Flutter documentation: Build apps for any screen. Google Software SDK. https://docs.flutter.dev/

Google Material Design. (2026). Material Design 3 system specification. Google Design Resources. https://m3.material.io/

GitHub. (2026). GitHub Copilot documentation and pair-programming guides. GitHub Docs. https://docs.github.com/copilot

Humphrey, W. S. (1995). A Discipline for Software Engineering. Addison-Wesley Professional.

Humphrey, W. S. (2000). Introduction to the Team Software Process (TSPi). Addison-Wesley Professional.

IEEE. (1998). IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998). IEEE Computer Society.

IEEE. (1998). IEEE Recommended Practice for Software Design Descriptions (IEEE Std 1016-1998). IEEE Computer Society.

IEEE Computer Society. (2024). Guide to the Software Engineering Body of Knowledge (SWEBOK Guide V4.0).

ISO/IEC/IEEE. (2011). Systems and software engineering — Requirements engineering (ISO/IEC/IEEE Std 29148:2011).

Karpathy, A. (2025). “Vibe coding” and AI-assisted software generation [public discussion / social media].

Karpathy, A. (2026). From vibe coding to agentic engineering [video presentation].

Document control note

This refined edition preserves the original assessment weights, milestone deadlines, core technical requirements, TSP evidence requirements, and late-submission policy while improving wording, structure, traceability, and visual presentation.